

GYM MANAGEMENT SYSTEM | GymTech

Industrial Informatics

*A complete solution for gym administration, trainer management, memberships and member access.
-all in one place-*

Team

Bogdan Dóra
Team Leader



Salajanu Mircea Vasile
Frontend Developer



Simionca Alexandru Petru Florin
Backend Developer



Kovacs Erik
Full-Stack Developer



Luidort Zsuzsa
Tester



Problem & Objectives

Problem & Solution

The system helps organize and manage gym operations: **members, subscriptions, payments, attendance, and roles.**

It centralizes information, replacing manual records with a role-based access platform.



User Roles

The application includes four main user roles, each with a dedicated dashboard:

- Admin
- Receptionist
- Trainer
- Member



Admin



Receptionist



Trainer



Member

Main Objective

To create a functional and organized gym management platform with role-based access, database persistence, and connected frontend-backend functionality.

TECHNOLOGY STACK

 Angular | ASP.NET Core Web API | Entity Framework Core | SQL Server 

Technologies Used

Frontend

Angular was used to build the graphical user interface, including the login page, sign-up page and role-based dashboards.



Backend

ASP.NET Core Web API was used to implement the application logic, API endpoints, services and CRUD operations.



Database

SQL Server was used for persistent data storage, while **Entity Framework Core** was used for database communication.



Tools

Swagger for testing, **GitHub** for version control, and **Figma** for UI design.



INTEGRATED TECH STACK

Robust. Scalable. Maintainable.

A modern ecosystem ensuring high performance and security for gym management.

Application Architecture

Key benefits

- This structure keeps the application organized, maintainable and easier to extend.

Layered structure (Frontend, Backend, Data, Database)

The application follows a layered architecture, separating the system into frontend, backend, data access and database layers.

Presentation in Angular (Pages & Dashboards)

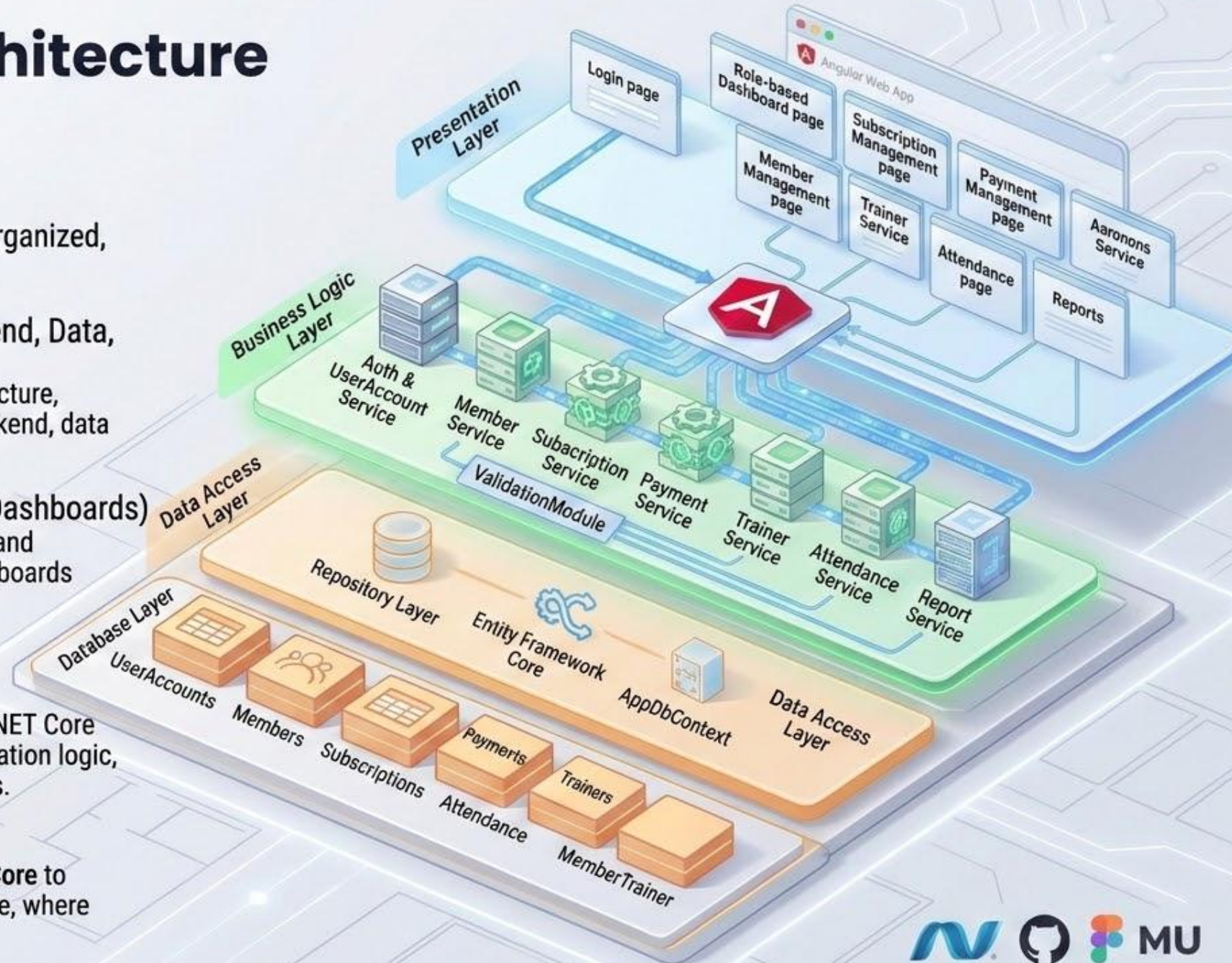
The frontend is implemented in Angular and contains the login page, role-based dashboards and main application pages.

Logic in ASP.NET Core API (Services & Validations)

The backend is implemented using ASP.NET Core Web API and contains the services, validation logic, authentication flow and CRUD operations.

Data in SQL Server via EF Core

The application uses **Entity Framework Core** to communicate with a SQL Server database, where all system data is stored persistently.



Use Case Diagram – User Roles & Functionalities

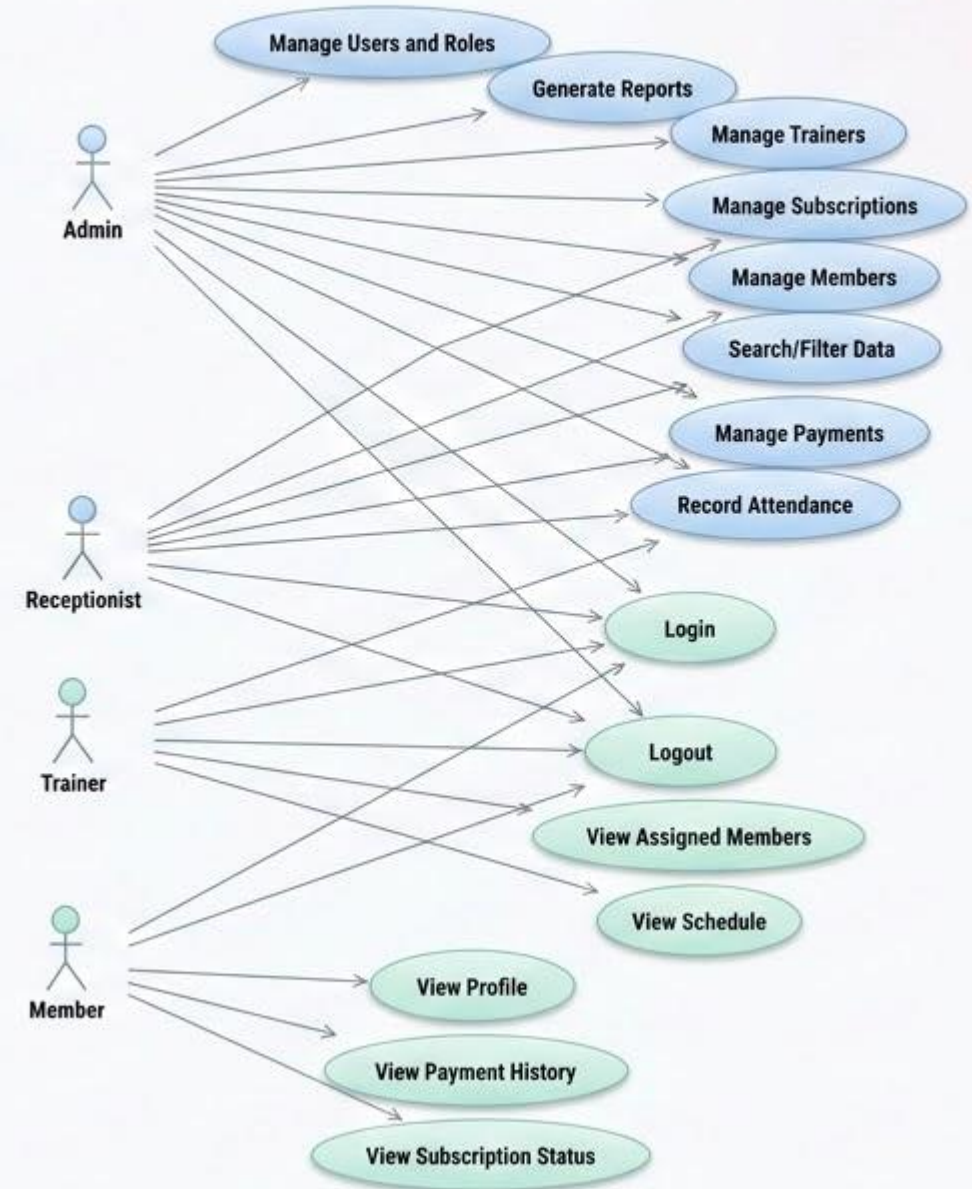
The system is based on four distinct roles, ensuring each has access only to the necessary functionalities.

The system is based on four roles: **Admin**, **Receptionist**, **Trainer** and **Member**.

Each role has access only to the functionalities needed for its activity inside the gym.

- **Admin:** manages the entire system
- **Receptionist:** manages members, subscriptions, payments and attendance
- **Trainer:** views assigned members and schedule
- **Member:** views profile, subscription status and payment history

The Use Case Diagram shows the interaction between these users and the main system functionalities.



Admin: manages the entire system



Trainer: views assigned members and schedule



Receptionist: manages members, and attendance



Member: views profile, subscription status and payment history

Frontend Implementation

The frontend of the application was developed using **Angular** and represents the graphical user interface of the Gym Management System.

It includes the main pages used by the system users: **login page, sign-up page, and role-based dashboards.**

The interface follows a modern design with a **dark theme, pink accent colors**, and a clear sidebar navigation.

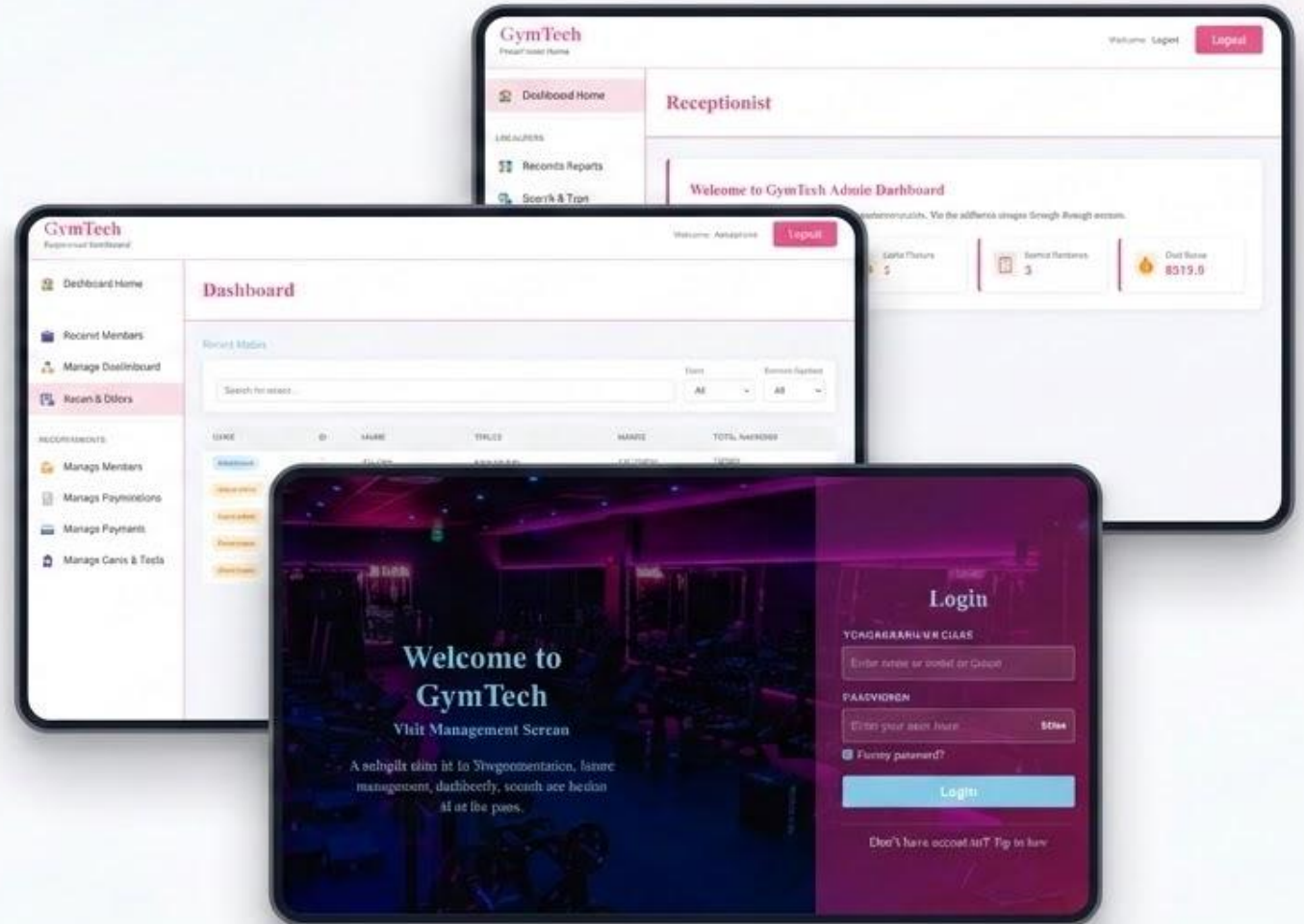
Each role has a dedicated dashboard with specific menu options and information relevant to that user type.

Implemented frontend pages:

- Login/Sign-up page
- Admin dashboard
- Receptionist dashboard
- Trainer dashboard
- Member dashboard

Goal:

To provide a clear and easy-to-use interface that allows users to access the correct features based on their role.



Backend Implementation

The backend of the application was developed using **ASP.NET Core Web API** and represents the logic layer of the Gym Management System.

It is responsible for processing requests from the frontend, applying validation rules, and communicating with the database.

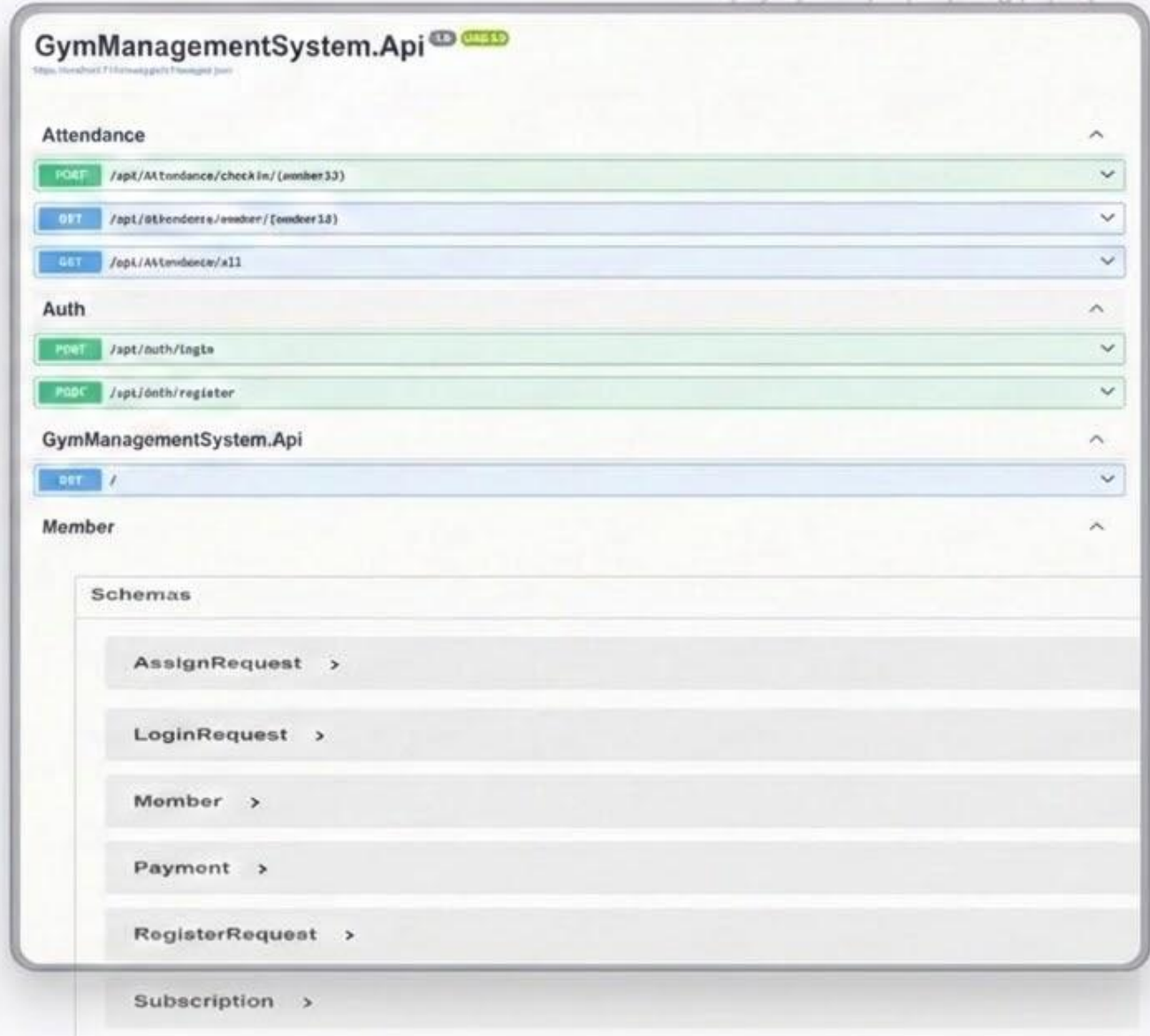
The backend includes **controllers, services, models, and database context classes**, which help keep the application organized and maintainable.

Main backend responsibilities:

- Implement API endpoints
- Manage CRUD operations
- Handle business logic
- Validate input data
- Communicate with **SQL Server** through **Entity Framework Core**
- Support authentication and role-based access

Goal:

To provide a reliable backend layer that connects the Angular frontend with the SQL Server database through secure API endpoints and business logic.



Class Diagram: Key Entities of GymTech

The class diagram presents the main entities of the Gym Management System and the relationships between them.

Main classes:

UserAccount

Stores login credentials and user roles, such as Admin, Receptionist, Trainer and Member.

Member

Stores member information and connects the member to subscriptions, payments, attendance records and trainers.

Subscription

Represents a member's gym subscription, including type, price, duration and validity period.

Payment

Stores payment information and links each payment to a member.

Attendance

Records member check-ins and attendance history.

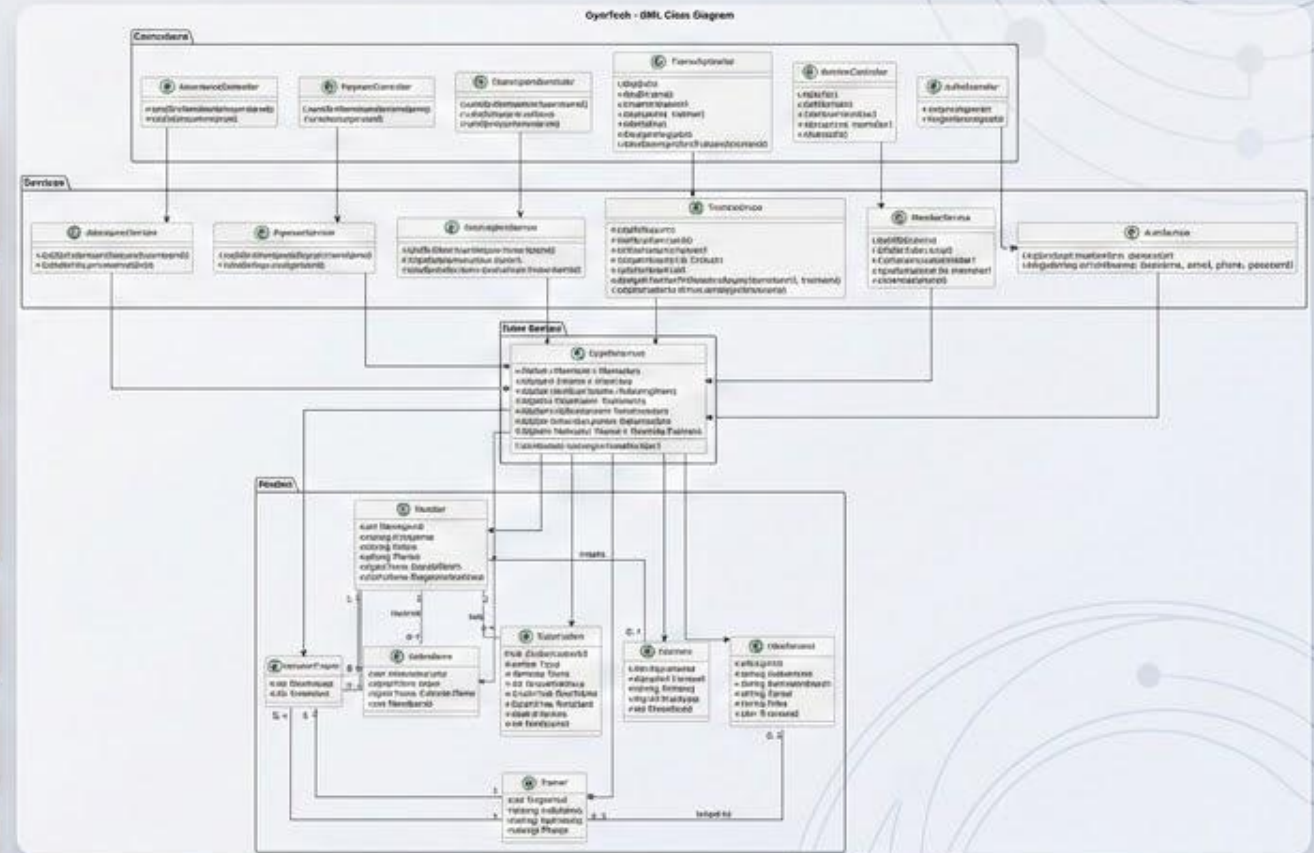
Trainer

Stores trainer information, such as name, phone and specialization.

MemberTrainer

Handles the many-to-many relationship between members and trainers.

The class structure keeps the application organized around clear responsibilities and supports the main OOP model of the project.



Database Structure / ERD

The database was designed in **SQL Server** and stores the main information required by the Gym Management System.

The system uses several connected tables to keep data organized and persistent.

Main database tables:

- ✓ **UserAccounts** – stores login information and user roles
- ✓ **Members** – stores gym member details
- ✓ **Trainers** – stores trainer information and specialization
- ✓ **Subscriptions** – stores subscription type, price, duration and validity
- ✓ **Payments** – stores payment records linked to members
- ✓ **Attendance** – stores member check-in history
- ✓ **MemberTrainer** – connects members and trainers

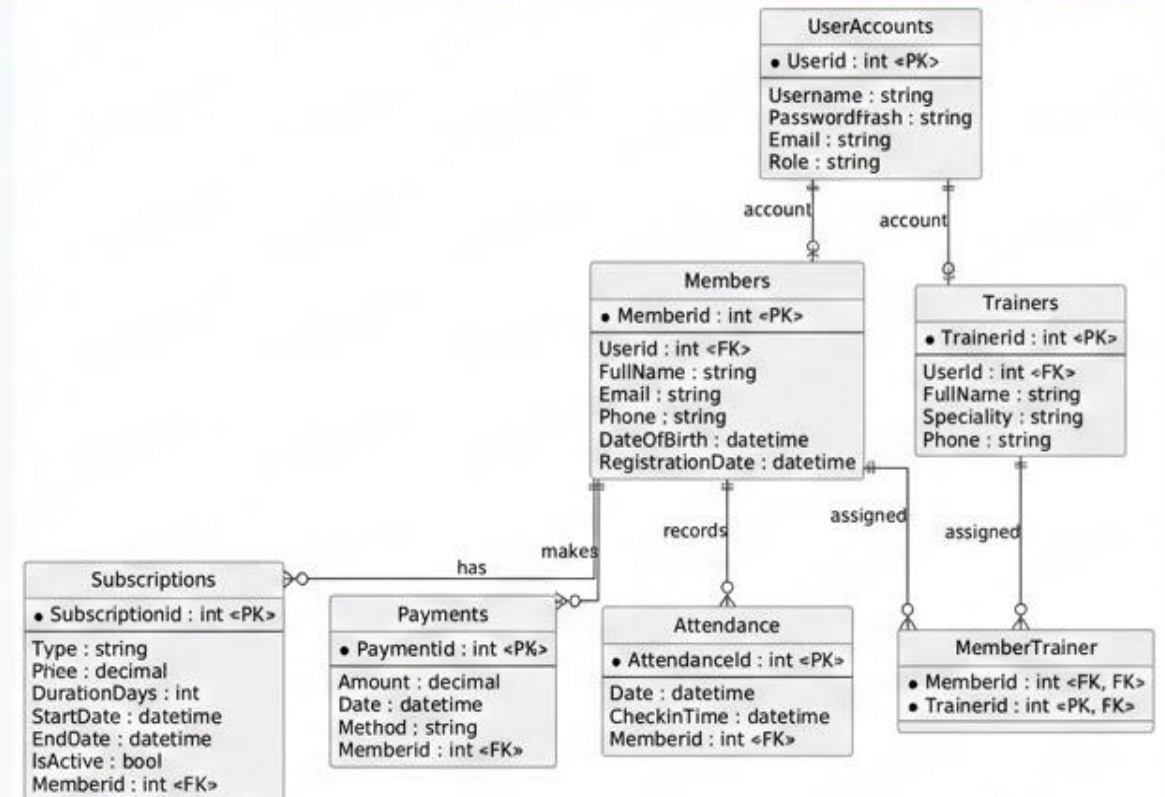
Relationships:

A member can have multiple subscriptions, payments and attendance records.

Members and trainers are connected through the **MemberTrainer** table.

Goal:

To ensure persistent data storage and clear relationships between the main entities of the application.



Frontend - Backend Integration

The full-stack part focused on connecting the **Angular frontend** with the **ASP.NET Core Web API backend**.

The frontend sends requests to the backend through API endpoints, while the backend processes the data, applies validation rules, and communicates with the SQL Server database.

A key part of the integration was the **authentication flow** and the **role-based dashboard redirection**.

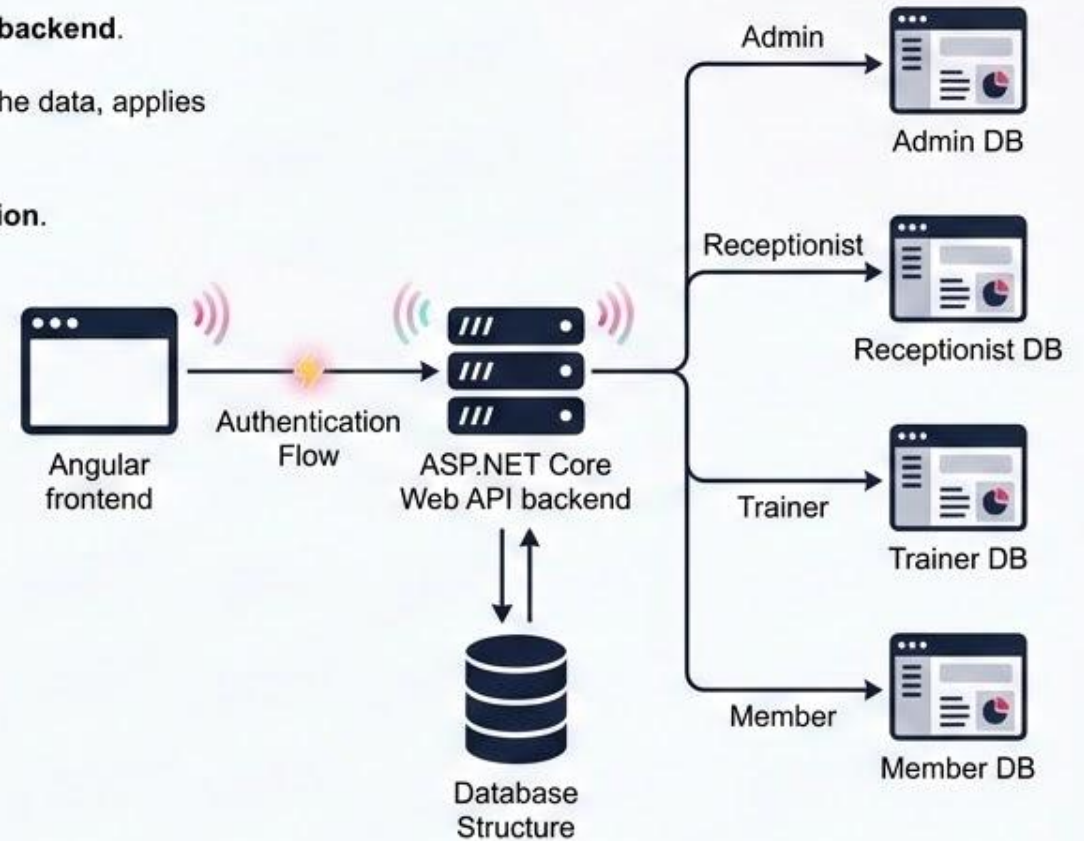
After login, the system checks the user role and redirects the user to the correct dashboard.

Role-based redirection:

- **Admin** → Admin Dashboard
- **Receptionist** → Receptionist Dashboard
- **Trainer** → Trainer Dashboard
- **Member** → Member Dashboard

Goal:

To make the frontend, backend, and database work together as one functional application.



Live Application Demo



Testing Strategy

The testing phase focused on verifying that the main workflows of the Gym Management System work correctly and that the frontend, backend, and database communicate properly.

Testing was performed using a combination of **manual testing**, **End-to-End testing**, **Swagger API testing**, and **database verification**.

Testing methods:

Frontend Testing

Checking login, sign-up, dashboard navigation, required fields, and general interface behavior.

End-to-End Testing

Testing complete user workflows from the interface, such as login, role-based navigation, and dashboard access.

Backend & API Testing

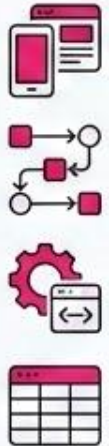
Verifying API endpoints, CRUD operations, service logic, and validation rules using Swagger.

Database Verification

Checking that created or modified data is stored correctly in SQL Server.

Goal:

To confirm that the application is stable, functional, and consistent across the main user workflows.



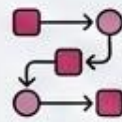
Testing Results

The main functionalities of the application were tested and verified successfully.

Validated workflows:



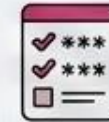
Login and sign-up forms



End-to-End login workflow



Role-based dashboard redirection



Required field validation



Member management operations



API responses in Swagger



Database persistence in SQL Server



Communication between frontend and backend

E2E testing confirmed that:

The main user flows work correctly from the interface to the backend and database.

Result:

The testing process confirmed that the application is functional, stable for the main workflows, and aligned with the core project requirements.

Conclusions & Future Improvements

The project achieved its main objective: developing a functional **Gym Management System** with role-based access, database integration, and connected frontend-backend functionality.

Main results

-  Functional Angular user interface
-  ASP.NET Core Web API backend
-  SQL Server database integration
-  Role-based dashboards
-  CRUD operations for core entities
-  End-to-End and manual testing of main workflows

Future improvements

-  Subscription email notifications
-  Advanced reports
-  Extended payment and subscription features
-  Improved security
-  More automated tests

Thank you for your attention!

